Member B (Unique) — Architecture: Bronze → Silver → Gold

You own: the medallion pipeline, the shared feature function, the loader, validation and the README. **Hand this whole PDF to your Claude session and say:** "*Read this document, then work through it step by step. Explain each step to me in 2–3 plain lines before moving on, and keep NOTES.local.md updated.*"

> The **Shared Contract** is attached at the end of this PDF (also in `docs/team/SHARED_CONTRACT.md`). It defines every table, column and function signature you must produce — **follow it exactly**, C and A code against it. Your files: `src/__init__.py`, `src/pipeline.py`, `src/features.py`, `src/loader.py`, `tests/test_features.py`, `reports/validation_output.txt`, `README.md`. Reference only: `hackathon-kit/src/pipeline.py` (copy ideas, never edit it).

Step 1 — Bronze + silver on a tiny sample (13:30 → 14:15)

A's real data arrives ~14:15. Don't wait: hand-make `data/_sample/` with 5 transformers (a few rows per file, in the Annex B columns + extras, include one duplicate and one lowercase ID) and build against it. **Delete `data/_sample/` once real data lands.**

1. `src/pipeline.py` — sections in this order, one status line printed per stage:
   - `load_bronze()` — read `data/<table>.csv` + `data/ingested/<table>__*.csv` **as text** (`dtype=str`), add `_ingested_at`, `_source`, write `bronze_*` (Contract §5).
   - `validate()` — HCL §4.5 checks as `assert`s, results written to `reports/validation_output.txt`: every transformer in operation is in the asset list (`customers_served`) · gases > 0 and in ppm (after the unit fix) · no test date in the future · every transformer has a customer type · costs and budget > 0 · row counts match `data/data_card.json` (skip with a printed note if the card is absent — judges' data).
   - `build_silver()` — IDs `strip().upper()`, drop exact duplicates, duplicate keys → keep latest `_ingested_at`, cast types, **unit-jump fix** (a test whose gases are ≥ 200× that transformer's median → divide by 1000, set `unit_fixed = 1`), missing `rating_class` → `'medium'` + `rating_class_assumed = 1`, missing `event` column → `'none'`. **Gaps stay NULL.**
2. **Re-runnable:** silver and gold are rebuilt in full every run (`if_exists="replace"`), so running twice gives identical tables. **Atomic:** write gold tables inside one `engine.begin()` transaction — and keep the kit's lesson: create the empty tables first, then append (see `hackathon-kit/PREFLIGHT.md` §2).

Commit suggestion: `git commit -m "add bronze and silver layers with validation"`.

Step 2 — Gold + the shared feature function (14:15 → 15:00) — the most important step

1. `src/features.py`: `FEATURE_COLUMNS`, `build_features(conn, snapshot_dates)`, `build_labels(conn, snapshot_dates)` — **exactly** as Contract §6–7. Use SQL with window functions where it is clearest:
   - latest usable test before a snapshot → `ROW_NUMBER() OVER (PARTITION BY transformer_id ORDER BY test_date DESC)` filtered to `test_date < snapshot_date` and after the last repair/failure
   - previous test for the deltas → `LAG()` over the same partition
   - 12-month load / temperature → `AVG`, `COUNT(CASE WHEN top_oil_temp_c > 80 …)`, p95 in pandas (`quantile(0.95)`)
   Comment each query with one line saying what it returns.
2. Gold tables (Contract §5): `dim_transformer` (surrogate key, `impact_weight = type_weight × customers`), `dim_date`, `fact_oil_test`, `fact_operation`, `fact_snapshot` (features + label for snapshots 2024-01-01 … 2026-10-01), plus `dim_action` (from `silver_costs`, surrogate `action_key`) and **empty** `dim_run`, `fact_risk`, `fact_plan` for C and A to write into. Show the panel the **conformed-dimension matrix** (Contract §5): `dim_transformer` and `dim_date` are shared by every fact table.
3. `tests/test_features.py` (pytest):
   - **Leakage test:** for every `fact_snapshot` row, the latest test date / operation month used is `< snapshot_date` (HCL validation check).
   - A repaired transformer's gas features ignore tests from before the repair.
   - `failed_next_12m` is NULL for snapshots after 2025-10-01.
4. **15:00 — push.** Tell C: "`fact_snapshot` *is on main.*" Suggested: `git commit -m "add gold star schema and snapshot features"`.

Done when: `python -m src.pipeline` prints row counts per table, `fact_snapshot` has ~4,800 rows (400 × 12 snapshots), and the tests pass.

Step 3 — Loader, validation output, README (15:00 → 15:40)

1. `src/loader.py`: `load_csv(table, csv_path)` (Contract §7) + CLI `python -m src.loader --table oil_tests --file path.csv`. Reject rows missing required Annex B columns or with an unknown transformer id; count them in `rows_rejected`. Copy accepted file to `data/ingested/`, re-run the pipeline. A's `/ingest` calls this. **Late-drop rule (HCL §4.6):** a renamed column must load without code changes → keep a small `COLUMN_ALIASES = {"old_name": "annex_b_name"}` dict at the top of `loader.py` (configuration, not code).
2. `README.md` (graded): architecture diagram (the medallion strip), setup and run steps, sample `curl` commands (from A), assumptions (incl. the mentor's inspection answer and our 1% new-unit risk), limitations, edge cases and how each is handled, AI-usage disclosure, team contribution table. Collect sections from A and C.

What the others are building

- **A — Data & Serving:** `data/*.csv` at ~14:15 (spec `docs/DATA_SPEC.md`), then the API calling your `build_features()` / `load_csv()`, Streamlit, Docker.
- **C — Model & Plan:** reads your `fact_snapshot`, trains, writes `models/`; `src/plan.py` writes into your `dim_run` / `fact_risk` / `fact_plan`.

You must be able to explain (Q&A is individual)

Each medallion layer and why it exists · the grain of `fact_snapshot` and how leakage is prevented · the window-function queries · the unit-jump fix and the repair reset · why star over 3NF, full refresh over incremental · why the transaction is split into create-then-append.

Shared Contract — read by every team member and every Claude session

> This is the single source of truth that lets three people build in parallel. **If your code and this file disagree, this file wins.** If you need to change something here (a column, a function signature), **tell the other two humans first** — never change it silently.

---

1. The problem in 4 lines

A utility has ~400 transformers and money to fix only a few. We **predict each transformer's chance of failing in the next 12 months** (calibrated, with an uncertainty range), compute **risk = probability × impact** (hospital 5, industry 3, urban 2, rural 1, × customers served), and choose **replace / inspect / nothing** within a budget with a **knapsack**, beating an age-based plan. Served through FastAPI + Streamlit + Docker. Full HCL guide: `Problem_Statements/Transformer_Fleet_Health_and_Replacement_Planning_Participant_Guide_DS_ML.docx`.

Mentor answer (12:00, recorded): an inspection tells us how severe the transformer really is; if it is worse than the model predicted, its risk should go up. → See §8 for how that drives the plan.

---

2. Rules for every Claude session (non-negotiable)

1. **Never run a git command that writes** (`add`, `commit`, `push`, `pull`, `checkout`, …). Suggest it; the human runs it. No `Co-Authored-By` / tool trailers in suggested messages.
2. **Never write inside** `hackathon-kit/` (read-only reference). Root `CLAUDE.md` applies.
3. **The HCL guide overrides** `CLAUDE.md`**'s stack list.** Docker, FastAPI, scikit-learn model training and pytest tests are **required** by HCL — use them.
4. **Only edit the files your member owns** (§3). Need something from another member's file? Ask the human. This is what keeps `git pull` free of conflicts.
5. **Readable over clever.** Every line must be explainable by a student under questioning. Comment every non-obvious query and every ML step with one plain-English line.
6. **Libraries (this is the list we told the panel):** pandas, numpy, sqlalchemy, scikit-learn, scipy, joblib, matplotlib, fastapi, uvicorn, pydantic, python-multipart, streamlit, requests, pytest, httpx. **Nothing else** without asking. Only `streamlit` and `python-multipart` still need `pip install`.
7. **Python package style:** `src/__init__.py` exists; import as `from src.features import build_features`; run modules as `python -m src.pipeline` from the repo root.
8. **Seeds:** `SEED = 42` everywhere randomness appears.

---

3. Repo layout and ownership

Path	Owner	What
`scripts/generate_data.py`	**A**	synthetic data generator (spec: `docs/DATA_SPEC.md`)
`data/*.csv`, `data/late_drop/`, `data/data_card.json`, `data/DATA_CARD.md`	**A**	generated data + data card
`src/__init__.py`	**B**	empty file
`src/pipeline.py`	**B**	bronze → silver → gold, validation. `python -m src.pipeline`
`src/features.py`	**B**	the **shared feature function** (training *and* API use it) + labels
`src/loader.py`	**B**	CLI + function to load a judge CSV (`/ingest` calls it)
`src/model.py`	**C**	baselines, models, validation, calibration, intervals, `score()`
`src/plan.py`	**C**	risk, knapsack plan, age-based plan
`src/api.py`	**A**	FastAPI app — thin, calls B's and C's functions

`ui/app.py`	**A**	Streamlit UI (3 pages)
`tests/test_features.py`	**B**	leakage + validation tests
`tests/test_plan.py`, `tests/test_model.py`	**C**	budget never exceeded, one action per unit, …
`tests/test_api.py`	**A**	endpoints respond in the contract shape
`models/`	**C** (generated)	`model.joblib`, `model_card.json`
`reports/experiments.csv`, `reports/evaluation.md`	**C** (generated)	experiment log, evaluation report
`reports/validation_output.txt`	**B** (generated)	output of the validation step
`Dockerfile`, `docker-compose.yml`, `requirements.txt`	**A**	packaging
`README.md`	**B**	graded; A and C send their sections to B
`warehouse.db`	generated	SQLite warehouse — **gitignored**, rebuilt by the pipeline
`NOTES.local.md`	each person, own laptop	private, gitignored

4. Source files (produced by A, consumed by B)

Full detail in `docs/DATA_SPEC.md`. Summary: **Annex B columns are mandatory.** The *extra* ones are ours and may be missing in judges' files, so **B's pipeline must cope without them.**

File	Columns
`customers_served.csv`	`transformer_id, customer_type, customers, region` + *extra* `rating_class`
`operation.csv`	`transformer_id, month, load_pct, top_oil_temp_c, age_years` + *extra* `event` (`none` / `failure` / `repair`)
`oil_tests.csv`	`transformer_id, test_date, h2_ppm, ch4_ppm, c2h2_ppm, c2h4_ppm, c2h6_ppm, co_ppm`
`costs_budget.csv`	`action, rating_class, cost, annual_budget`

Data period: monthly 2023-10-01 → 2026-09-01. "Today" = 2026-10-11.

5. Warehouse tables (`warehouse.db`, SQLite) — built by B

Bronze — raw, never edited

Rebuilt on every pipeline run from **files on disk**: `data/<table>.csv` plus every file in `data/ingested/<table>__*.csv` (what `/ingest` saved). The files are the immutable replay tape.

Table	Columns
`bronze_customers_served`, `bronze_operation`, `bronze_oil_tests`, `bronze_costs_budget`	all source columns **as text, exactly as received** + `_ingested_at` (TEXT ISO) + `_source` (file name)

Silver — validated, cleaned, typed, one row per business key

Table	Key	Columns
`silver_transformer`	`transformer_id`	`customer_type, customers INT, region, rating_class, rating_class_assumed INT` (1 if extra column missing → `medium` used)
`silver_operation`	`transformer_id, month`	`load_pct REAL, top_oil_temp_c REAL, age_years REAL, event TEXT` (`'none'` if column absent)
`silver_oil_test`	`transformer_id, test_date`	`h2_ppm … co_ppm REAL` (all ppm), `unit_fixed INT` (1 if a ppb→ppm ×1000 jump was corrected)
`silver_costs`	`action, rating_class`	`cost REAL`
`silver_budget`	—	`annual_budget REAL` (one row; latest file wins)

Cleaning rules: IDs `strip().upper()`; exact duplicates dropped; duplicate keys → keep the latest `_ingested_at`; gaps stay **NULL** (never filled in silver).

Gold — star schema, shaped for reading

Table	Grain (one row = …)	Columns
`dim_transformer`	one transformer	`transformer_key INT` (surrogate), `transformer_id, customer_type, customers, type_weight INT, impact_weight REAL` (= type_weight × customers), `region, rating_class, rating_class_assumed`
`dim_date`	one calendar day	`date_key INT` (YYYYMMDD), `date, year, quarter, month, is_snapshot_date INT`
`fact_oil_test`	one transformer × one test	`transformer_key, date_key`, six gases, `tdcg` (sum of the six)
`fact_operation`	one transformer × one month	`transformer_key, date_key, load_pct, top_oil_temp_c, age_years, is_failure INT, is_repair INT`
`fact_snapshot`	**one transformer × one quarterly snapshot date**	`transformer_key, transformer_id, snapshot_date`, **all** `FEATURE_COLUMNS` **(§6)**, `failed_next_12m INT` (NULL = not labelable yet)
`dim_run`	one scoring / plan run	`run_id TEXT` (key), `created_at, kind ('score'/'plan'), as_of, budget, model_version`
`dim_action`	one action × size class	`action_key INT` (surrogate), `action ('replace'/'inspect'), rating_class, cost` — from `silver_costs`
`fact_risk`	one run × one transformer	`run_id, transformer_id, failure_prob_12m, prob_low, prob_high, status, health_score, impact_weight, risk_score, risk_low, risk_high, model_version`
`fact_plan`	one run × one chosen action	`run_id, transformer_id, action_key, action, cost, risk_before, risk_removed`

`dim_run`, `fact_risk`, `fact_plan` rows are **written by C's code** (via `src/plan.py` / `src/model.py`); B creates the empty tables and fills `dim_action`. Storing every run lets us show *what changed* after the late drop.

Shared (conformed) dimensions: one dimension serves several fact tables. This is what we told the panel:

Fact table ↓ / Dimension →	`dim_transformer`	`dim_date`	`dim_run`	`dim_action`
`fact_oil_test`	✓	✓ test date		
`fact_operation`	✓	✓ month		
`fact_snapshot`	✓	✓ snapshot date		
`fact_risk`	✓	✓ as-of date	✓	
`fact_plan`	✓	✓ as-of date	✓	✓

Customer type → weight (hospital 5, industry 3, urban 2, rural 1) is kept **inside** `dim_transformer` (`type_weight`) rather than in its own table: that is the star-schema choice (one join, a little repetition) over a snowflake.

Snapshot dates: first day of each quarter, **2024-01-01** → **2026-10-01**. **Labelable** only if `snapshot_date + 12 months ≤ 2026-10-01` → snapshots 2024-01-01 … 2025-10-01 have labels; later ones have `failed_next_12m = NULL` (used for scoring only). 2026-10-01 is "now".

---

6. The feature list — defined ONCE in `src/features.py` (owner B)

```
FEATURE_COLUMNS = [
    "age_years", "load_mean_12m", "load_p95_12m", "temp_mean_12m", "hot_months_12m",
    "has_oil_test", "n_tests", "months_since_last_test",
    "h2_last", "ch4_last", "c2h2_last", "c2h4_last", "c2h6_last", "co_last",
    "tdcg_last", "tdcg_delta", "c2h2_delta",
    "ratio_c2h2_c2h4", "ratio_ch4_h2", "ratio_c2h4_c2h6",
    "months_since_repair",
]
```

Every feature uses **only data dated strictly before** `snapshot_date` (this is the leakage rule).

Feature	Definition

`age_years`	from the latest `silver_operation` month before the snapshot
`load_mean_12m`, `load_p95_12m`	mean / 95th percentile of `load_pct` over months in [snapshot − 12 months, snapshot)
`temp_mean_12m`	mean `top_oil_temp_c`, same window
`hot_months_12m`	count of months in the window with `top_oil_temp_c > 80`
`has_oil_test`	1 if any usable test before the snapshot, else 0
`n_tests`	number of usable tests before the snapshot
`months_since_last_test`	(snapshot − last test date) in days / 30.44; NULL if no test
`*_last`	the six gases of the latest usable test; NULL if none
`tdcg_last`	sum of the six gases (total dissolved combustible gas) of the latest test
`tdcg_delta`, `c2h2_delta`	(last − previous) ÷ months between them; NULL if < 2 usable tests
`ratio_c2h2_c2h4`, `ratio_ch4_h2`, `ratio_c2h4_c2h6`	gas ratios of the latest test, denominator + 0.1 to avoid ÷0; NULL if no test
`months_since_repair`	months since the latest `repair` **or** `failure` event before the snapshot; NULL if none

"Usable test" = a test **after** the most recent repair/failure event before the snapshot (a repair resets the gas pattern — edge case E5). **Missing stays NULL** — handling missing values is the model's job (C).

Label: `failed_next_12m = 1` if `silver_operation` has `event = 'failure'` in a month **m** with `snapshot_date ≤ m < snapshot_date + 12 months`; `0` if not; `NULL` if not labelable (§5).

7. Function interfaces — the hand-off points

```
# src/features.py  (B)
FEATURE_COLUMNS: list[str]
def build_features(conn, snapshot_dates: list[str]) -> pd.DataFrame:
    """Columns: transformer_id, snapshot_date, *FEATURE_COLUMNS. Reads silver tables only."""
def build_labels(conn, snapshot_dates: list[str]) -> pd.DataFrame:
    """Columns: transformer_id, snapshot_date, failed_next_12m."""

# src/pipeline.py  (B)
DB_PATH = "warehouse.db"
def get_connection(): ...                    # sqlite3 / SQLAlchemy connection to DB_PATH
def run_pipeline() -> dict:                  # rebuild bronze → silver → gold; returns row counts per table

# src/loader.py  (B)
def load_csv(table: str, csv_path: str) -> dict:
    """table ∈ {oil_tests, operation, customers_served, costs_budget}.
    Checks required Annex B columns, copies the file to data/ingested/<table>__<timestamp>.csv,
    re-runs the pipeline. Returns {"rows_loaded": int, "rows_rejected": int, "status": "ok"|"error", "detail":
str}."""

# src/model.py  (C)
MODEL_PATH = "models/model.joblib"; CARD_PATH = "models/model_card.json"
def train() -> dict:                         # reads fact_snapshot, runs the full evaluation, saves model +
card + reports
def score(features: pd.DataFrame) -> pd.DataFrame:
    """In: output of build_features. Out per transformer: transformer_id, failure_prob_12m, prob_low, prob_high,
    status, health_score, top_drivers (list of {"feature","contribution"}), data_quality, model_version."""
def model_info() -> dict:                    # contents of model_card.json

# src/plan.py  (C)
TYPE_WEIGHTS = {"hospital": 5, "industry": 3, "urban": 2, "rural": 1}
def compute_risk(scores: pd.DataFrame, dim_transformer: pd.DataFrame) -> pd.DataFrame:  # adds impact_weight,
risk_score, risk_low, risk_high
def make_plan(risk: pd.DataFrame, costs: pd.DataFrame, budget: float) -> dict:
    """{"actions": [{"transformer_id","action","cost","risk_before","risk_removed","rationale"}],
        "total_cost", "budget", "risk_before", "risk_removed", "risk_reduced_pct"}"""
def age_plan(risk: pd.DataFrame, costs: pd.DataFrame, budget: float) -> dict:          # same shape, replaces
oldest first
```

Status values (HCL §6.2): `scored` · `scored_low_confidence` (no oil test, last test > 18 months old, or only 1 test) · `insufficient_data` (no operation rows) · `invalid_input` (API only) · `plan` (API `/plan` only).

---

8. Decision logic — how replace vs inspect is decided (owner C)

1. **Model** gives each transformer `p` (calibrated) and a range `[p_low, p_high]`. Few/old/no tests → wide range.
2. **Value of each action** (risk removed):
   - **Replace:** `(p − 0.01) × impact`. A replacement removes all the risk except a new unit's own, which we assume is a 1% yearly failure chance.
   - **Inspect:** `(p_high − p) × impact`. An inspection removes a **stated fraction** of the risk, and that fraction is **how uncertain we are about the unit**: the hidden risk an inspection could reveal. Wide range (we know little) → inspection is valuable. Narrow range (we already know) → it is worth about 0. *Why not one fixed fraction (e.g. 30%)?* Inspection costs ~1/23 of a replacement, so with a fixed fraction the optimiser inspects almost every unit and replaces almost none. Tying the fraction to uncertainty sends inspections to where they actually tell us something.
   - **Rule:** a transformer with **no usable oil test or a last test > 18 months old cannot be replaced blind** — only inspected (don't spend ~350k on something we have not looked at).
3. **Knapsack** picks at most one action per transformer, maximising total risk removed, total cost ≤ budget (dynamic programming over cost in units of 5,000 — all costs are multiples of 5,000).
4. **Inspection feedback loop (mentor's point):** an inspection *is* a new oil test. Its result arrives through `/ingest` (`oil_tests`) → pipeline rebuilds → model re-scores → if severe, `p` goes **up** and the range **narrows** → replace value rises, inspect value falls → **the next plan replaces it** (if budget allows). If the test is clean, `p` goes down and it drops off the list. The late-drop file (new oil tests) demonstrates exactly this. **A human engineer approves every plan** (`requires_approval: true`).

8b. Assumptions the panel told us to make (12:00 design review)

Costs and budget are **ours to assume**. We assume these values and write them into the README "Assumptions" section:

Action	small	medium	large
replace	150,000	350,000	700,000
inspect	10,000	15,000	25,000

Annual budget **5,000,000**; late-drop rehearsal budget **4,000,000** (a 20% cut). This is already in `costs_budget.csv` (DATA_SPEC §3.4).

8c. Health index: a score for each field (owner C, in `src/model.py`)

We told the panel each field gets its own score. This is that rule. It is used three ways: (1) as the `health_score` returned by the API, (2) as the **rule-based baseline** the ML model must beat, and (3) as an **explanation** ("gas score 30 because acetylene is at condition 3"). Gas limits follow the **IEEE C57.104 dissolved-gas "condition" levels** (C1 normal → C4 severe). **Verify the numbers against the standard before quoting them.**

Field	Condition 1 (normal)	C2	C3	C4 (severe)
H2 (ppm)	≤ 100	101–700	701–1800	> 1800
CH4	≤ 120	121–400	401–1000	> 1000
C2H2	≤ 1	2–9	10–35	> 35
C2H4	≤ 50	51–100	101–200	> 200
C2H6	≤ 65	66–100	101–150	> 150
CO	≤ 350	351–570	571–1400	> 1400

Score component	Points deducted from 100
Gas score: worst gas condition	C1 → 0 · C2 → 15 · C3 → 30 · C4 → 40
Trend score: `tdcg_delta` (ppm per month)	≤ 10 → 0 · 10–30 → 10 · > 30 → 20
Load score: `load_p95_12m`	≤ 90 → 0 · 90–100 → 10 · > 100 → 20
Age score: `age_years`	≤ 20 → 0 · 20–30 → 6 · 30–40 → 12 · > 40 → 20

`health_score = 100 − (gas + trend + load + age)`, clipped to 0–100. **NULL when there is no usable oil test**, because the gas and trend scores cannot be computed, and an untested unit must never look healthy. The model's *probability* is still produced for such units (from age, load and `has_oil_test`), flagged low confidence.

---

9. API (owner A) — which function each endpoint calls

Endpoint	Calls
`POST /predict` (optional `transformer_ids`, `as_of`)	`build_features(conn, [as_of or "2026-10-01"])` → `score()` → `compute_risk()`; writes `fact_risk`
`POST /plan` (`budget`, optional costs)	latest `fact_risk` → `make_plan()` + `age_plan()`; writes `fact_plan`
`POST /ingest` (multipart CSV + `{"table": ...}`)	`load_csv()`
`GET /model/info`	`model_info()`
`GET /explain/{transformer_id}`	`score()` for that unit → `top_drivers` + its latest tests/operation rows
`GET /health`	DB reachable? model file loaded?

Response shapes: HCL guide §6.1 (`predictions[]` with `failure_prob_12m`, `prob_interval_90`, `health_score`, `impact_weight`, `risk_score`, `top_drivers`, `data_quality`, `status`, `model_version`).

---

10. Timeline and sync points (code freeze 16:15)

Time	A — Data & Serving	B — Architecture (Unique)	C — Model & Plan
13:15–13:30	read docs, set up	read docs, set up	read docs, set up

13:30–14:15	**generator + data card**	bronze + silver on a 5-row hand-made sample	`plan.py` + its tests (toy data)
14:15 sync	**push** `data/`	pull data, run on real files	—
14:15–15:00	`api.py` with stub responses in contract shape	gold + `features.py` + labels + leakage test	`model.py` on a stand-in `fact_snapshot`
15:00 sync	—	**push** `fact_snapshot` **code**	pull, train on real data
15:00–15:40	wire API to real functions; Streamlit; Dockerfile	`loader.py`, validation output, README	evaluation, calibration, model card, experiment log
15:40 sync	`docker compose up` works	README sections merged	**push** `models/` **+** `reports/`
15:40–16:10	rehearse demo end-to-end, fix, tag `final`		

Cut list if behind (in this order): Streamlit to one page → bootstrap intervals from 20 not 50 models → drop the `/explain` sensor detail. **Never cut:** time-based grouped validation, calibration, the knapsack, the API.

---

11. Git workflow (the humans run these, not Claude)

- Everyone works on `main`, touching only their own files (§3).
- Before pushing: `git pull --rebase` then `git push`. Commit **at least every 30–45 minutes**, small messages.
- Commit `data/` and `models/` (judges and Docker need them). Never commit `warehouse.db`, `.env`, `NOTES.local.md`.